Linux Driver III

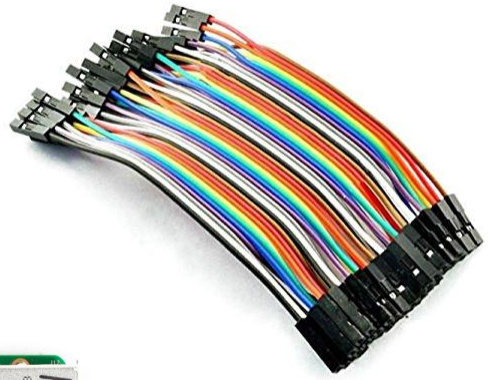
開發學生: 李瑞恆、陳品翰
高效能計算實驗室

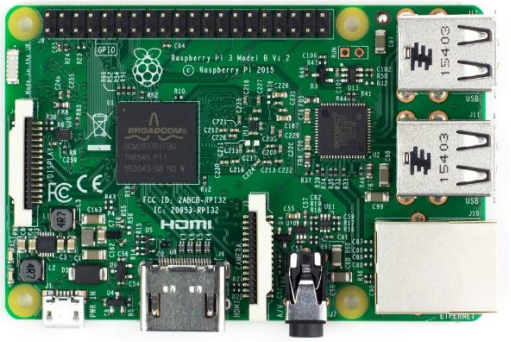
中正大學 資訊工程學系

Linux Driver III

- 用GPIO控制四位數七段顯示器
- 材料:







七段顯示器簡介

- 型號:tm1637



CLK 時鐘訊號 (連接GPIO)

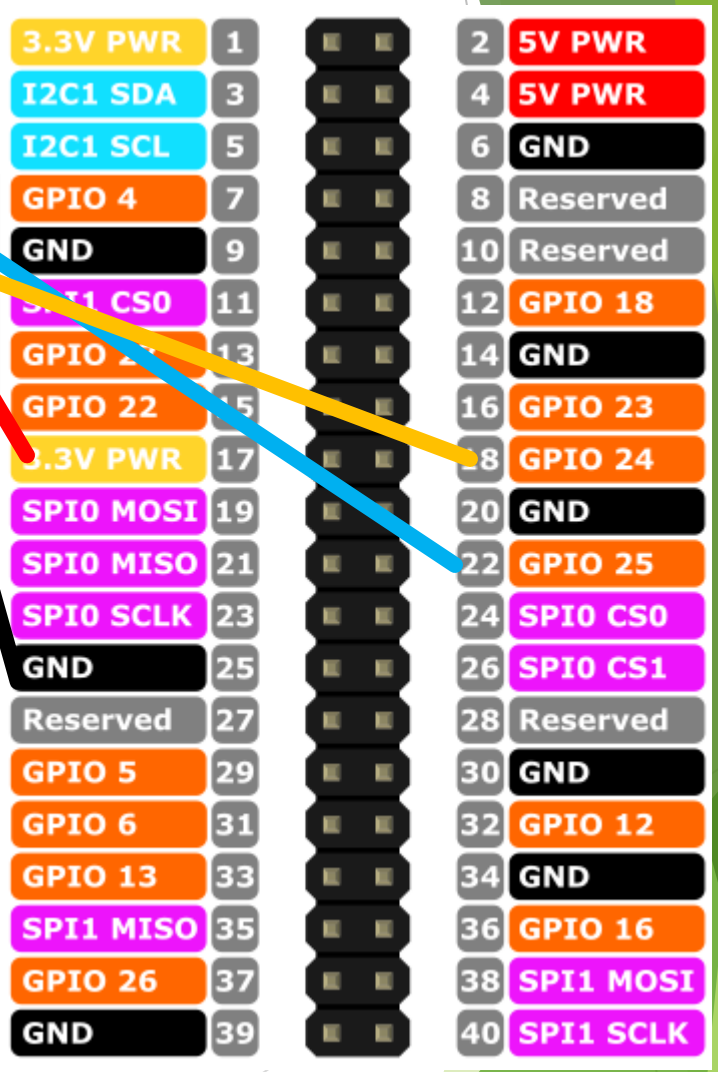
DIO 數據輸入輸出腳 (連接GPIO)

VCC 供電 (連接任一PWR)

GND 接地 (連接任一GND)

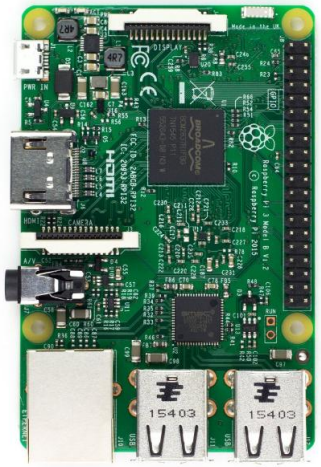
Connection Example





Remember which GPIO pins you plugged in!

CLK	GPIO 25
DIO	GPIO 24
VCC	3.3V PWR
GND	GND



How to control it?

- Using C's file operations (libgpiod)
- Let's see an example

```
1. int main() {
2.     initialize();
3.     showNum(9218);
4.     sleep(1);
5.
6.     clearDisplay();
7.     end();
8.     return 0;
9. }
```

How to control it?

- initialize:像上個實驗，首先開啟GPIO device，設置方向為輸出、CLK與DIO先設為0

```
#include <gpiod.h>

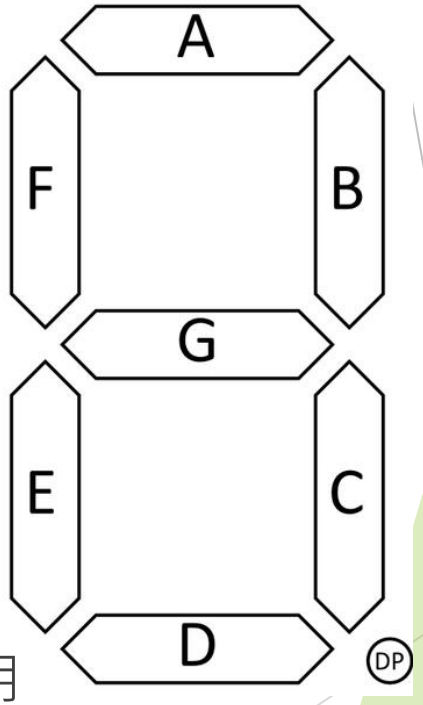
#define CHIPNAME "/dev/gpiochip0"
#define CLK_PIN 25
#define DIO_PIN 24
```

```
1.  void initialize() {
2.      chip = gpiod_chip_open(CHIPNAME);
3.      if (!chip) {
4.          perror("Open chip failed");
5.          return;
6.         }
7.      clk_line = gpiod_chip_get_line(chip, CLK_PIN);
8.      dio_line = gpiod_chip_get_line(chip, DIO_PIN);
9.      if (!clk_line || !dio_line) {
10.         perror("Get line failed");
11.         return;
12.     }
13.     gpiod_line_request_output(clk_line, "tm1637", 0);
14.     gpiod_line_request_output(dio_line, "tm1637", 0);
15. }
```

How to show Number?

- 先了解寫入的方法:
  每次輸入七段顯示器，以一個byte(8個bits)為最小單位
- 要顯示哪一段，

  該段的訊號就設為1。
- 範例: (會從A開始寫入)




```
//H,G,F,E,D,C,B,A
 {0,0,1,1,1,1,1,1}, //0
 {0,0,0,0,0,1,1,0}, //1
 {0,1,0,1,1,0,1,1}, //2
 {0,1,0,0,1,1,1,1}, //3
```

註:H段為中間冒號，只有輸入第二位數字時可用



基本function: writeByte過程

```
1.  void writeByte(uint8_t byte) {
2.      for (int i = 0; i < 8; i++) {
3.          writeCLK(0);              //CLK low
4.          writeDIO(byte & 0x01);    // set Data bits
5.          writeCLK(1);              // CLK high
6.          byte >>= 1;
7.      }
8.      /* Waiting for ACK
9.         每次輸入完 8 個 bits DIO 會回傳一個 ACK，我們下面不讀取 ACK，單純等待 ACK 完成 */
10.     writeCLK(0);
11.     writeCLK(1);
12.     writeCLK(0);
13. }
```

基本function: Start and Stop Signal

- 當CLK為1時，DIO從1變為0 表示Start (開始輸入)
- 當CLK為1時，DIO從0變為1 表示Stop (停止輸入)

```
1.  void startWrite() {     //send start signal to TM1637
2.      writeCLK(1);
3.      writeDIO(1);
4.      writeDIO(0);
5.  }
6.
7.  void stopWrite() {     //send stop signal to TM1637
8.      writeCLK(0);
9.      writeDIO(0);
10.     writeCLK(1);
11.     writeDIO(1);
12. }
```

Showing Number Procedure

- 有兩種模式，連續輸入四位數字、或只輸入某一位數字
- 這邊使用連續輸入四位數字的模式

```
#define AUTO_MODE {0,1,0,0,0,0,0,0} //0x40  (Command: auto increment mode)
#define FIXED_MODE {0,1,0,0,0,1,0,0} //0x44 (Command: fixed position mode)
```



Showing Number Procedure

- 有兩種模式，連續輸入四位數字、或只輸入某一位數字
- 這邊使用連續輸入四位數字的模式




```
1.  // 顯示一個四位數的數字
2.  void showNum(int num) {
3.      int digits[4] = {0};
4.
5.      // 個位、十位、百位、千位
6.      for (int i = 3; i >= 0; i--) {
7.          digits[i] = num % 10;
8.          num /= 10;
9.      }
10.
11.     // 設定寫入模式: 自動加位址
12.     startWrite();
13.     writeByte(AUTO_MODE); // 設定自動加位址
14.     stopWrite();
15.
16.     // 設定寫入起始位址 0xC0
17.     startWrite();
18.     writeByte(START_ADDR); // 位址 0 開始
19.     for (int i = 0; i < 4; i++) {
20.         writeByte(segment_map[digits[i]]);
21.     }
22.     stopWrite();
23.
24.     // 設定亮度最大 & 顯示開啟
25.     startWrite();
26.     writeByte(DEFAULT_BRIGHTNESS | 0x07); // 顯示開啟 + 最高亮度
27.     stopWrite();
28. }
```

...

回到本範例的main來看:

```
1. int main() {
2.     initialize();
3.     showNum(9218);
4.     sleep(1);
5.
6.     clearDisplay();
7.     end();
8.     return 0;
9. }
```



clearDisplay()跟showNum一樣
只是每位數字writeByte都寫入
{0,0,0,0,0,0,0,0}
(表示把全部bits清空不亮)

```
1. void end() {
2.     gpiod_line_release(clk_line);
3.     gpiod_line_release(dio_line);
4.     gpiod_chip_close(chip);
5. }
```

(關閉GPIO)

DEMO

1. 請依照你所接的接腳，完成example.c的 initialize()、end()、writeDIO()、writeCLK()、clearDisplay()這些function。
2. 基本題: 請從1000倒數到0，每秒更新一次、一次減100。(1000, 900, 800, … , 0) 結束後將顯示器清空(熄滅不亮)。

注意: 範例程式皆可自行修改或優化，亦可自行撰寫。